



UNIVERSITY OF LEEDS

Formal Language & Finite Automata

Regular Expressions

Dr Noleen Koehler

University of Leeds

All reading is from the "Introduction to the theory of computation" -
Micheal Sipser

- Regular expression - Pages 63 - 76 (essential)

Regular expressions

Regular expressions

A subset of the regular expression notation,

- **[]** A bracket expression. Matches a single character that is contained within the brackets.
- **+** Greedily matches the expression to its left 1 or more times.
- ***** Greedily matches the expression to its left 0 or more times.
- **A | B** Matches expression A or B. If A is matched first, B is left untried. or more times

Examples

- **[a-z]⁺** matches any string of lowercase characters of length greater than 0.
- **[a-c]^{*}[1-9—d-f]⁺** matches any string that starts with 0 or more character from the set 'a', 'b' or 'c', followed by one or more characters from the number 1 to 9, or one or more characters from the set 'd', 'e' or 'f'.

Regular expressions

Regular expressions are used commonly in programming and in Unix for input validation, searching strings and find/replace.

Definition (Regular Expressions)

R is a **regular expression** if R is

1. $\{a\}$ for some $a \in \Sigma$,
2. $\{\epsilon\}$,
3. $\emptyset$,
4. $(R_1 \cup R_2)$, where R_1 and R_2 are regular expressions,
5. $(R_1 \circ R_2)$, where R_1 and R_2 are regular expressions,
6. (R_1^*) , where R_1 is a regular expression.

The notation R^+ is shorthand for $R \circ R^*$, that is, one or more repetitions of R . Note that $R^+ \cup \{\epsilon\} = R^*$.

Theorem

A language is regular if and only if some regular expression describes it.

This is, clearly, a bidirectional statement. That is, if the theorem is correct then the following Lemmas are true.

Lemma

If a language is described by a regular expression, then it is regular.

and

Lemma

If a language is regular, then some regular expression describes it.

Regular expressions

We construct a non-deterministic finite automaton from a regular expression by considering each of the cases in the definition of regular expressions.

1. If $R = a$ for some $a \in \Sigma$ then $L(R) = \{a\}$. The following non-deterministic finite automaton recognises $L(R)$.



Formally, we construct the NFA $N = (\{q_0, q_1\}, \Sigma, \delta, q_0, \{q_1\})$ where $\delta(q_0, a) = q_1$ and $\delta(r, b) = \emptyset$ for $r \neq q_0$ or $b \neq a$.

2. If $R = \epsilon$ then $L(R) = \{\epsilon\}$.



Formally, we construct the NFA $N = (\{q_0\}, \Sigma, \delta, q_0, \{q_0\})$ where $\delta(r, b) = \emptyset$ for any r and b .

3. If $L(R) = \emptyset$, we construct the NFA $N = (\{q_0\}, \Sigma, \delta, q_0, \emptyset)$ where $\delta(r, b) = \emptyset$ for any r and b .



4. Let $R = R_1 \cup R_2$. Regular languages are closed under the union operation.
5. Let $R = R_1 \circ R_2$. Regular languages are closed under the concatenation operation.
6. Let $R = R_1^*$. Regular languages are closed under the Kleene star operation.

Lemma

If a language is regular, then some regular expression describes it.

Given a regular language, we must demonstrate the existence of a regular expression that describes the language.

- As the language is regular there exists a DFA that recognises.
- We convert the DFA into a GNFA
- We apply an iterative argument to "shrink" the GNFA to an equivalent GNFA with 1 fewer states.
- When only 2 state exist in the GNFA then the label on the transition between them in the regular expression that describes the regular language.